

Smali By bithowl: Chapter 6 Smali Naming System

("The Secret Code Every Android Reverse Engineer Must Learn")



SMALI BY BITHOWL : CHAPTER 6

SMALI NAMING SYSTEM

"The Secret Code Every Android Reverse Engineer Must Learn"

```
String → Ljava/lang/String;
int → I
boolean → Z
```



THE STORY (HOOK)

Airport → AP
Hospital → HSP
University → UNI

What looks like random letters... becomes a language your mind learns automatically.

Smali shortens data types into compact descriptors to make bytecode smaller, faster and easier for Android to process.

 Learn the abbreviations. Read the language.

WHY DOES SMALI USE DESCRIPTORS?

- Android apps can contain thousands of classes and millions of references.
- Descriptors make the DEX file:
 - Smaller
 - Faster to parse
 - Easy for the runtime to interpret
 - Consistent across every Android application

Java / Kotlin Source Code → **DEX (Compact Descriptors)**

OVERVIEW: WHAT YOU WILL LEARN

- Primitive Type Descriptors
- Object Descriptors
- Class Descriptors
- Array Descriptors
- Method Signatures
- How to Read Them Instantly



1 PRIMITIVE TYPE DESCRIPTORS

Java Type	Smali Descriptor	Example in Smali
int	I	const/4 v0, I
boolean	Z	const/4 v0, Z
byte	B	const/4 v0, B
char	C	const/16 v0, C
short	S	const/16 v0, S
long	J	const-wide/16 v0, J
float	F	const/high16 v0, F
double	D	const-wide/high16 v0, D
String	Ljava/lang/String;	const-string v0, "Hello"
void	V	return-void

 Most descriptors are the first letter of the type. Exceptions: boolean → Z, long → J

2 OBJECT DESCRIPTORS

General syntax: `Lpackage/ClassName;`

Examples:

String → Ljava/lang/String;
Intent → Landroid/content/Intent;
ArrayList → Ljava/util/ArrayList;
File → Ljava/io/File;

Rules to remember:

- Begin with L
- Uses / instead of .
- Ends with ;



3 CLASS DESCRIPTORS

Identifies the complete location of a class.

Example:

Java: `com.example.app.LoginActivity`
Smali: `Lcom/example/app/LoginActivity;`

Breakdown:

`L` + `com/example/app/` + `LoginActivity` + `;`

Start of object | Package name (/ instead of .) | Class name | End of descriptor

 Whenever Smali references a class, it almost always uses this format.

4 ARRAY DESCRIPTORS

Syntax: `[I` means array

Java	Smali	Meaning
<code>int[]</code>	<code>[I</code>	Array of int
<code>boolean[]</code>	<code>[Z</code>	Array of boolean
<code>String[]</code>	<code>[Ljava/lang/String;</code>	Array of String
<code>Intent[]</code>	<code>[Landroid/content/Intent;</code>	Array of Intent
<code>String[][]</code>	<code>[[Ljava/lang/String;</code>	2D Array of String
<code>int[][]</code>	<code>[[[I</code>	3D Array of int

 Add more [for more dimensions.



5 METHOD SIGNATURES (BASICS)

General syntax: `methodName(parameters) returnType`

Examples:

Java	Smali	Breakdown
<code>void login()</code>	<code>login()V</code>	No params, returns void
<code>int add(int a, int b)</code>	<code>add(II)I</code>	Two int params, returns int
<code>boolean isOK()</code>	<code>isOK()Z</code>	No params, returns boolean

Inside the parentheses = parameters
After parentheses = return type

6 MORE METHOD SIGNATURE EXAMPLES

Java Method	Smali Signature
<code>boolean isLoggedIn()</code>	<code>isLoggedIn()Z</code>
<code>String getToken()</code>	<code>getToken()Ljava/lang/String;</code>
<code>void send(Intent intent)</code>	<code>send(Landroid/content/Intent;)V</code>
<code>String login(String user, String password)</code>	<code>login(Ljava/lang/String;Ljava/lang/String;)Ljava/lang/String;</code>
<code>int[] getScores()</code>	<code>getScores()[I</code>
<code>List<String> getList()</code>	<code>getList()[Ljava/util/List;</code>

 The return type always comes after the closing).

7 READING DESCRIPTORS LIKE A REVERSE ENGINEER

Example Smali Method:

```
.method public verifyToken(Ljava/lang/String;)Z
```

Access Modifier	Method Name	Parameters	Return Type
public	verifyToken	1. Ljava/lang/String; → 2. I → int	Z → boolean

8 BUG HUNTER PERSPECTIVE — WHY DESCRIPTORS MATTER

 String IDs Expose

- Hidden API Endpoints
- Hardcoded Credentials
- Firebase URLs
- Encryption Keys
- Debug Messages
- Feature Flags

 Method IDs Help Identify

- Cryptographic Functions
- Authentication Routines
- Root Detection Methods
- SSL Pinning Logic
- Biometric Authentication
- Network Callers

 Class Definitions Reveal

- Exported Activities
- Internal Services
- Utility Classes
- Network Clients
- Database Managers
- Background Tasks

 Code Items Contain

- Actual Instructions
- Execution Flow
- Logic Paths
- Try-Catch Blocks
- Branches & Loops
- Behavior Decisions

Before reading Smali, know where to look. Structure reveals purpose.

9 ACCESS MODIFIERS

 public	Accessible from anywhere
 private	Accessible only within the class
 protected	Accessible to subclasses
 static	Belongs to class (not object)
 final	Cannot be overridden or modified
 abstract	No implementation (must be overridden)

 Modifiers appear in .class, .field, and .method directives.

10 HOW EVERYTHING FITS TOGETHER

Think of every Smali file as a blueprint.

```
.class
.super
.implements
.field
.method
Instructions
.end method
```



11 PRACTICAL EXERCISE

- Extract any APK using APKTool
`apktool d app.apk`
- Locate a .smali file.
- Identify the structure:

`.class` → `.super` → `.source`
`.implements` → `.field`
`.methods` → constructors
inner classes (\$)

Try Translating:

```
.method private process(Landroid/content/Intent;Ljava/lang/String;)V
```

Can you identify?

- Method name
- Parameters
- Return type



12 QUICK RECAP

- DEX uses compact descriptors to save space and improve performance.
- Primitive types use single letters: I, Z, J, etc.
- Objects begin with L and end with ;.
- Arrays begin with [and support multiple dimensions.
- Method signatures encode parameters inside () and return type after.
- Understanding descriptors helps you read Smali without a decompiler.



13 FINAL THOUGHT

To a beginner, `Ljava/lang/String;` looks like noise. To an Android reverse engineer, it's simply **String**.

The symbols never changed. Only your ability to read them did.

Before you understand what a program does, you must first understand how it's organized.

By - bithowl



Open any Smali file, and you'll immediately notice something strange.

Instead of seeing: String

You see: Ljava/lang/String;

Instead of: int

You see: I

Instead of: boolean

You see: Z

At first, it looks like random letters.

Like some secret language Android invented just to confuse developers. But it's actually a carefully designed naming system called **Type Descriptors**.

Once you learn it, you'll be able to read almost every Smali method, field, and class without needing a decompiler. Think of this chapter as learning Android's alphabet.

The Story (Hook)

Imagine landing in a new country where every road sign is abbreviated.

Instead of: Airport

You see: AP

Instead of: Hospital

You see: HSP

At first, nothing makes sense. But after a few days, your brain automatically translates every abbreviation.

Smali works exactly the same way.

Android shortens data types into compact descriptors to make bytecode smaller and faster to process. Once your brain learns these descriptors, you'll stop translating them consciously.

You'll simply read them.

Why Does Smali Use Descriptors?

Android applications can contain:

- Thousands of classes
- Tens of thousands of methods
- Hundreds of thousands of variables

If every instruction repeatedly stored long class names like: java.lang.String

The DEX file would become unnecessarily large. Instead, Android uses compact descriptors.

These descriptors are:

- Smaller
- Faster to parse
- Easy for the runtime to interpret
- Consistent across every Android application

You'll encounter them everywhere in Smali.

Primitive Type Descriptors

Primitive Java types are represented using a single character.

Modifier	Meaning
public	Accessible from anywhere
private	Accessible only within the class
protected	Accessible to subclasses
static	Belongs to the class rather than an object
final	Cannot be overridden or modified
abstract	Method or class has no implementation

Notice something interesting. Most descriptors are the first letter of the Java type. But there are exceptions.

For example: boolean → Z , long → J

These choices come from the JVM descriptor specification that Android also follows. They may seem unusual at first, but you'll recognize them quickly with practice.

Object Descriptors

Primitive types use one letter. Objects use a different format.

General syntax: Lpackage/ClassName;

Example:

Java: String

Smali: Ljava/lang/String;

Another example:

Java: Intent

Smali: Landroid/content/Intent;

Notice three rules:

- Every object begins with **L**
- Package separators use **/**
- Every descriptor ends with **;**

Once you recognize this pattern, object descriptors become easy to read.

Class Descriptors

A class descriptor identifies the complete location of a class.

Example:

Java: `com.example.app.LoginActivity`

Smali: `Lcom/example/app/LoginActivity;`

Breaking it down:

L

Start of object.

`com/example/app/`

Package name.

`LoginActivity`

Class name.

;

End of descriptor.

Whenever Smali references a class, it almost always uses this format.

Array Descriptors

Arrays introduce one additional symbol.

The character: **[** means **array**.

Examples:

int[] → **[I**

boolean[] → **[Z**

String[] → **[Ljava/lang/String;**

Intent[] → **[Landroid/content/Intent;**

Multiple dimensions simply add more brackets.

Example: `String[][]`

becomes: `[[Ljava/lang/String;`

The number of [characters tells you the number of dimensions.

Method Signatures

Methods have their own descriptor format.

General syntax: methodName(parameters)returnType

Example:

Java: void login()

Smali: login()V

Here:

- () means no parameters.
- V means the method returns void.

Another example:

Java: int add(int a, int b)

Smali: add(II)I

Breaking it down:

- First I → first parameter (int)
- Second I → second parameter (int)
- Final I → return type (int)

The return type always appears **after** the closing parenthesis.

More Method Signature Examples

1)

Java: boolean isLoggedIn()

Smali: isLoggedIn()Z

2)

Java:String getToken()

Smali: getToken()Ljava/lang/String;

3)

Java: void send(Intent intent)

Smali: send(Landroid/content/Intent;)V

4)

Java: String login(String user, String password)

Smali: login(Ljava/lang/String;Ljava/lang/String;)Ljava/lang/String;

Although these signatures may look intimidating at first, they're simply compact descriptions of method parameters and return types.

Reading Descriptors Like a Reverse Engineer

Suppose you encounter: **.method public verifyToken(Ljava/lang/String;I)Z**

Without seeing any Java code, you already know:

- Method name: verifyToken
- First parameter: String
- Second parameter: int
- Returns: boolean
- Access modifier: public

That's the power of understanding descriptors.

Bug Hunter Perspective

At first glance, descriptors look like strange symbols. For a bug hunter, they're valuable clues. Imagine you're searching for authentication logic.

Instead of reading thousands of methods manually, you can quickly recognize signatures such as:

authenticate(Ljava/lang/String;Ljava/lang/String;)Z

Immediately you know:

- Two String parameters
- Returns boolean

That likely represents a login or credential validation method.

Similarly, descriptors help identify:

- Methods accepting Intent objects
- Network-related classes
- Cryptographic APIs
- Database operations
- File access routines
- Authentication callbacks

During reverse engineering, researchers often search for specific descriptors to locate security-sensitive code much faster than reading every class line by line.

Learning this notation transforms what looks like random characters into meaningful information.

Practical Exercise

Extract any APK using APKTool.

Open a Smali file and locate:

- A field descriptor
- A class descriptor
- A method signature

Try translating them back into Java without using JADX.

For example: `.method private process(Landroid/content/Intent;Ljava/lang/String;)V`

Can you identify:

- Method name?
- Parameters?
- Return type?

Repeat this exercise across several classes.

Within a short time, you'll begin recognizing descriptors instinctively.

Quick Recap

- Smali represents Java types using compact descriptors.
- Primitive types use single-character symbols such as I, Z, and V.
- Objects begin with L and end with ;.
- Arrays begin with [.
- Class descriptors identify the full package and class name.
- Method signatures encode parameters and return types in a compact format.
- Understanding descriptors allows reverse engineers to interpret Smali without relying on decompiled Java.

Final Thought

To a beginner, `Ljava/lang/String;` looks like noise. To an Android reverse engineer, it's simply **String**.

The symbols never changed. Only your ability to read them did.

And that's one of the biggest milestones in learning Smali:

The moment Android's secret language stops looking like code...

...and starts looking like a conversation.

By - bithowl